SYNCHRONOUS FIFO
UVM VERIFICATION REPORT

by Sushant Gautam

SystemVerilog • UVM 1.2 • Synopsys VCS 2025.06

Project report based on the supplied testbench log and EPWave waveform

1. Project Overview

This project verifies a synchronous First-In First-Out (FIFO) using a SystemVerilog RTL design and a UVM-based verification environment. The testbench generates read and write transactions, drives the DUT through a virtual interface, observes the resulting activity, checks data ordering with a reference queue, and collects functional coverage.

The supplied project configuration identifies an 8-bit data path and an 8-entry FIFO. The simulation was run with Synopsys VCS X-2025.06-SP1 and UVM 1.2.

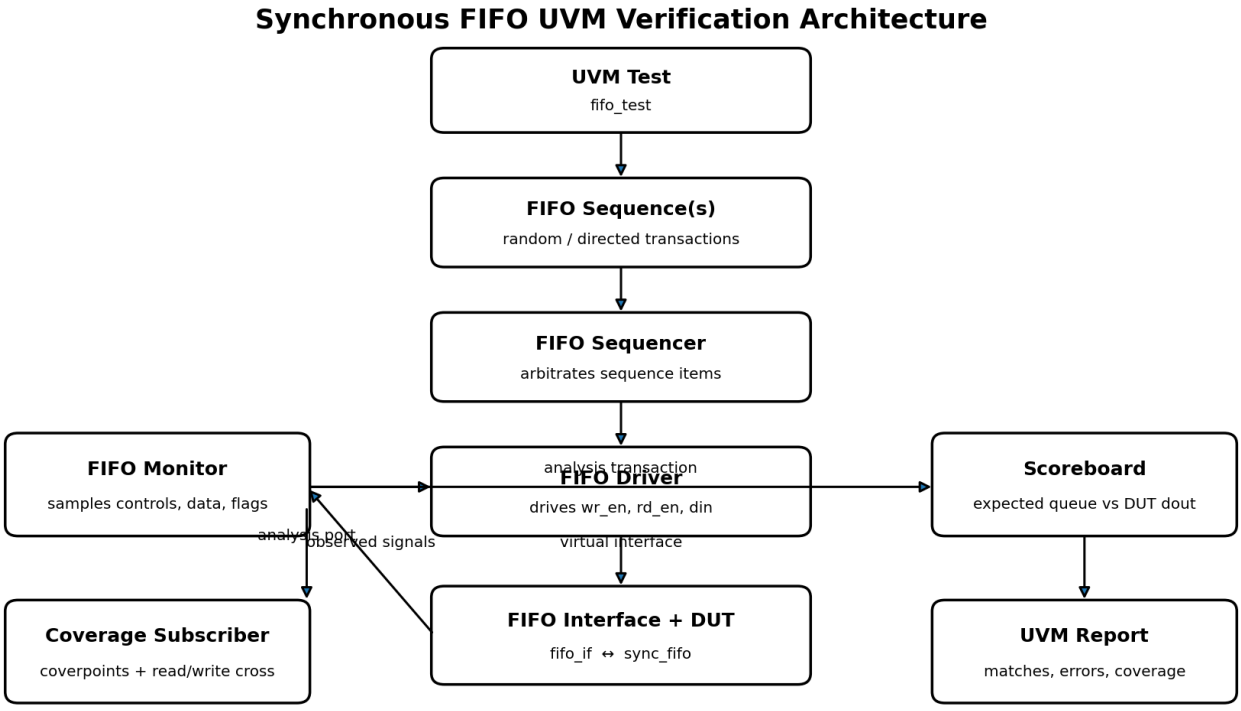
2. DUT Design Description

The synchronous FIFO stores input data on an accepted write and returns the oldest stored data on an accepted read. The interface signals visible in the supplied waveform are clk, rst_n, wr_en, rd_en, din[7:0], dout[7:0], full, and empty.

Signal	Direction relative to DUT	Purpose
clk	Input	Common synchronous clock
rst_n	Input	Active-low reset
wr_en	Input	Requests a write
rd_en	Input	Requests a read
din[7:0]	Input	Write data
dout[7:0]	Output	Read data
full	Output	Indicates FIFO cannot accept another write
empty	Output	Indicates FIFO has no valid stored data

3. UVM Testbench Architecture

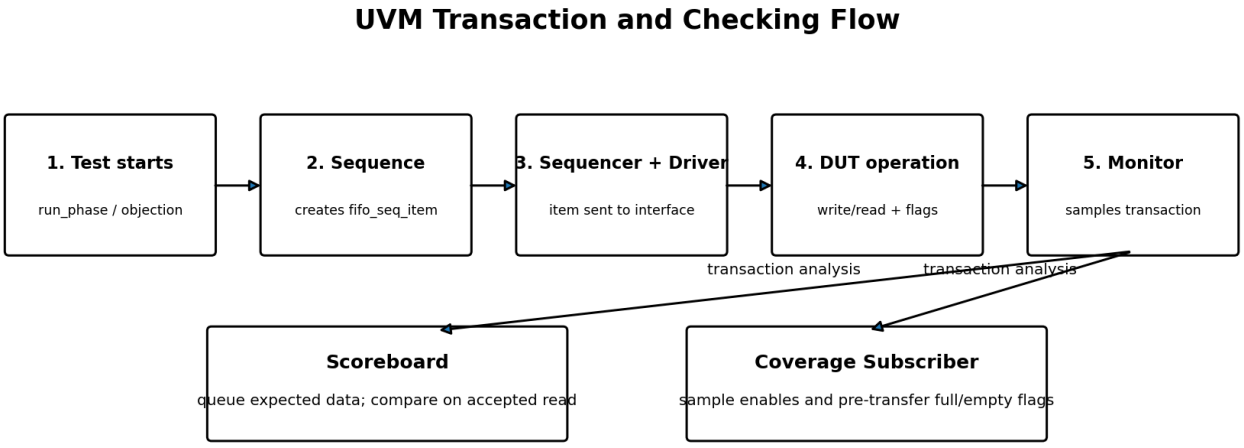
The UVM environment is modular. The test starts a sequence on the agent sequencer. The sequencer supplies sequence items to the driver. The driver drives the FIFO interface. The DUT performs the requested operation. The monitor observes the interface and publishes a transaction through its analysis port. That analysis stream is connected to both the scoreboard and the coverage subscriber.



4. UVM Components and Responsibilities

Component	Responsibility
fifo_seq_item	Transaction object containing randomized write/read enables and input data, together with observed output/status fields used by checking and coverage.
fifo_sequence	Creates and sends transactions. The shared sequence file contains random, single-write/read, fill, drain, and wraparound sequence classes; a class only runs when the test starts it.
fifo_sequencer	Arbitrates sequence items and provides them to the driver.
fifo_driver	Receives each item and drives wr_en, rd_en, and din onto the virtual FIFO interface.
fifo_monitor	Samples the interface at the clock edge. The described implementation records pre-transfer full/empty flags and captures post-update output/status after a small delay, then broadcasts the transaction.
fifo_agent	Groups the sequencer, driver, and monitor into one reusable UVM agent.
fifo_scoreboard	Receives monitor transactions and maintains an expected-data queue to check FIFO ordering and data integrity.
fifo_coverage	A uvm_subscriber that samples read/write enables and the pre-transfer full/empty status; reports coverpoint and cross coverage.
fifo_env	Creates the agent, scoreboard, and coverage component and connects the monitor analysis port to the scoreboard and coverage subscriber.
fifo_test	Creates the environment and starts the selected sequence during run_phase.

5. Transaction and Verification Flow



1. 1. The top-level testbench generates the clock and reset, instantiates the interface and DUT, places the virtual interface into the UVM configuration database, and calls run_test().
2. 2. UVM constructs fifo_test and its environment during the build/connect phases.
3. 3. During run_phase, the test raises an objection, creates the selected sequence, and starts it on env.agt.seqr.
4. 4. The sequence creates fifo_seq_item objects and uses start_item()/finish_item() to send each request.
5. 5. The driver obtains each item and drives the request signals onto fifo_if.
6. 6. At the active clock edge, the DUT accepts a write only when wr_en is asserted while not full, and accepts a read only when rd_en is asserted while not empty.
7. 7. The monitor samples the transfer and status. It publishes the observed transaction through its analysis port.
8. 8. The scoreboard updates its expected queue and compares expected data with the DUT output for accepted reads. Coverage samples the transaction fields.
9. 9. At the end of the test, report_phase prints the coverage and scoreboard summary; the test drops its objection and simulation finishes.

6. What the Scoreboard Compares

The scoreboard is a reference model for FIFO data ordering. It does not compare the DUT's internal memory array or pointer implementation. Instead, it independently predicts the sequence of data values that should be returned.

Expected queue operation:

- Accepted write: when wr_en=1 and full_before=0, the scoreboard pushes din into its expected queue.
- Accepted read: when rd_en=1 and empty_before=0, the scoreboard takes the oldest value from the expected queue and compares it against the monitor's observed dout.
- Rejected write: when wr_en=1 and full_before=1, the scoreboard must not add the input data to the expected queue.
- Rejected read: when rd_en=1 and empty_before=1, the scoreboard must not remove an item or perform a normal data comparison.

A matching read is evidence that the DUT returned the oldest expected value. A mismatch indicates possible data corruption, ordering error, incorrect read/write acceptance, or a monitor/scoreboard sampling problem.

7. Functional Coverage Model

The coverage subscriber receives the same monitor transactions as the scoreboard. Its reported coverage is based on the configured coverpoints and read/write cross. The full and empty coverpoints use the pre-transfer status fields, so the coverage records the FIFO status that existed before the current request was applied.

Coverage item	What it measures	Latest reported result
Write enable	Both write disabled and enabled values	100%
Read enable	Both read disabled and enabled values	100%
Full status	Both not-full and full values	50%
Empty status	Both not-empty and empty values	100%
Read/write cross	Combinations of read and write enables	100%
Overall functional coverage	Aggregate of the defined coverage items	90%

The missing full-status bin explains the 90% overall result in the supplied run: full coverage is only 50%, meaning only one of the two full-status bins was sampled. In this run, the full flag was not observed asserted by the coverage sample.

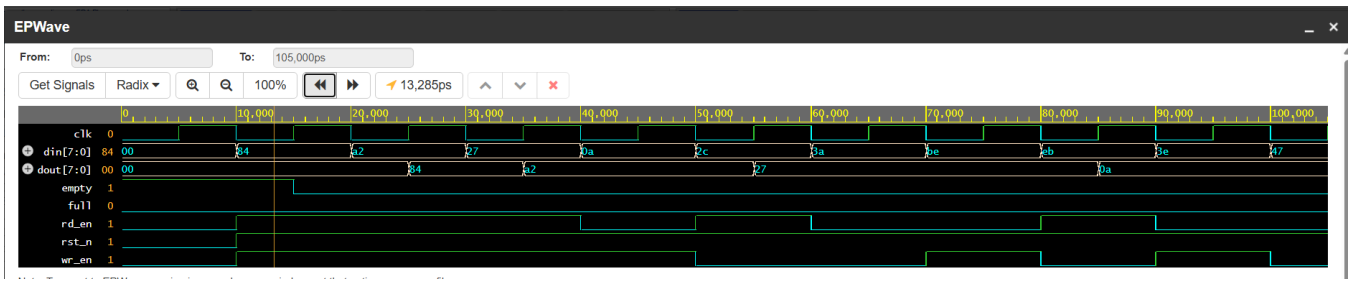
8. Simulation Results from the Supplied Log

Metric	Observed result
Simulator	Synopsys VCS X-2025.06-SP1
UVM library	UVM 1.2 (Synopsys build)
Simulation completion	Completed; $finish at 105,000 ps
UVM errors / fatals	0 / 0
UVM warnings	0 in report summary; compile-time argument-name warning remains
FIFO functional coverage	90.00%
Read/write cross coverage	100.00%
Write / read coverage	100.00% / 100.00%
Full / empty coverage	50.00% / 100.00%
Unread expected entries at end	2

The log shows successful data matches for the read values 0x84, 0xA2, 0x27, and 0x0A. It also reports two expected queue entries remaining when the test ended. Therefore, this run completed without UVM errors, but it did not finish with an empty reference queue.

9. Output Waveform

The following EPWave capture shows the top-level FIFO signals over the simulation interval. The displayed traces include clk, din, dout, empty, full, rd_en, rst_n, and wr_en.



Waveform observations from the supplied screenshot:

- rst_n is initially low and later deasserts high, releasing the DUT from reset.
- empty is asserted at the beginning and deasserts after data is written.
- din changes as write transactions are presented.
- dout updates on read activity and shows values such as 0x84, 0xA2, 0x27, and 0x0A.
- full remains low throughout this particular capture, consistent with the missing full-status coverage bin.

10. Recommended Next Improvements

10. Run a directed full-condition sequence: begin from an empty FIFO, perform eight accepted writes, then issue another write while full.
11. Ensure the monitor publishes a sample after the eighth write so full_before=1 is visible on the extra write attempt. Because the monitor samples pre-transfer flags, merely reaching full at the end of the last write may not hit the full bin.
12. Drain all eight stored entries after the full-condition test and confirm the scoreboard expected queue is empty.
13. Fix the compile-time method argument-name warning by naming the fifo_coverage::write argument t to match uvm_subscriber::write(T t).
14. Add assertions or explicit checks for overflow and underflow rejection if those behaviors are part of the DUT specification.
15. Keep separate test names for random traffic, fill/full, drain/empty, single write/read, and pointer wraparound so each scenario can be run and debugged independently.

11. Conclusion

The supplied run demonstrates a functioning UVM testbench structure: transactions are generated, driven to the DUT, observed by the monitor, checked by the scoreboard, and sampled by functional coverage. The scoreboard reports matching read data and the simulation ends with no UVM errors. The main remaining verification gaps in this run are the unhit full-status coverage bin and two unread expected queue entries. A directed full-and-drain scenario is the next logical step.